

Graduation Project Proposal Form

1. Project Information

Project Title: Polymind - An AI-Powered Workspace for Network Engineering

Course/Track: Computer Science - Software Engineering Track

Team Members:

[Student Name] -

[Student Name] -

[Student Name] -

[Student Name] -

[Student Name] -

[Student Name] -

[Student Name] -

2. Project Overview

Objective: To develop a full-stack SaaS workspace that helps network engineers use artificial intelligence to design, document, analyze, troubleshoot, and manage IT infrastructure. The platform provides authenticated workspaces, projects, AI conversations, and reusable engineering resources in one environment.

Scope of Work: The project includes a React and TypeScript web application, a Laravel REST and streaming API, PostgreSQL data storage, Redis-backed cache and queues, workspace and project management, secure user authentication, role-based access control, AI chat with file attachments, conversation history, an engineering dashboard, and a library for agents and prompt templates. The first release focuses on network engineering workflows while preserving a modular structure for future cloud, DevOps, and cybersecurity modules. Native mobile applications and hardware-device management are outside the scope of this release.

Expected Outcomes: A working Docker-deployable platform where users can register, sign in, create and manage workspaces and projects, upload supporting files, ask an AI assistant for network-engineering help through streamed responses, keep persistent conversation history, and view real usage information. The final system will include automated tests for key frontend and backend flows.

3. Problem Statement

Network engineers routinely switch between topology notes, device configuration references, troubleshooting records, documentation tools, and general-purpose AI chats. This fragments important context, makes collaboration and traceability difficult, and slows down repetitive engineering tasks. General AI tools also do not provide a dedicated workspace for organizing engineering projects, attached files, conversation history, access roles, and usage controls. There is a need for a secure,

integrated platform that brings these activities together while allowing engineers to use reliable AI providers through one consistent interface.

4. Proposed Solution

Technologies Used: Frontend: React 18, TypeScript, Vite, Tailwind CSS, Zustand, and Radix UI. Backend: PHP 8.3 with Laravel 12. Database: PostgreSQL with UUID-based records. Cache, queues, and sessions: Redis. Authentication and authorization: Laravel Sanctum, Socialite OAuth, and Spatie Permission. AI: a provider-agnostic layer supporting OpenAI-compatible providers, Anthropic, Gemini, Groq, OpenRouter, DeepSeek, Mistral, and Ollama, with retry and fallback handling. Real-time AI replies use Server-Sent Events. Development and deployment use Docker Compose, Nginx, and GitHub Actions.

System Architecture: The React client communicates with a versioned Laravel API. Sanctum protects user sessions and API requests, while the backend provisions organizations and workspaces, enforces ownership and role permissions, and stores users, projects, conversations, messages, uploaded files, usage records, agents, and templates in PostgreSQL. Redis supports caching, queued tasks, sessions, and rate limiting. The AI manager selects the configured provider, sends requests, streams assistant responses to the client when supported, records token usage and cost information, and applies retries or provider fallbacks when a request fails. Docker containers run the frontend, API, PostgreSQL, and Redis as a reproducible deployment stack.

5. Resources Needed

Hardware/Software: A development computer with Docker Desktop, internet access for configured AI providers, and optional cloud hosting for deployment. Required software includes PHP 8.3 and Composer, Node.js and npm, Docker Compose, PostgreSQL, Redis, Git and GitHub, and a modern web browser. The project may use Figma for interface planning and an AI provider account for live model access during evaluation.

6. Approval

Instructor/Advisor:

Signature: